INDUSTRIAL TRAINING REPORT

AI RESEARCH ASSISTANT

Submitted in partial fulfilment of the

Requirements for the award of

Degree of Bachelor of Technology in Computer Science & Engineering

Session: 2026-27

Submitted By

Name: Nishant Kumawat

University Roll No.: 23ESKCY036 **Semester/Branch:** 4th Year (IOT)

SUBMITTED TO:

Department of Computer Science & Engineering

SWAMI KESHVANAND INSTITUTE OF TECHNOLOGY, MANAGEMENT & GRAMOTHAN, JAIPUR

DECLARATION

I hereby declare that the Industrial Training Report entitled **"AI Research Assistant"** is an authentic record of the work carried out during my Industrial Training from **15 May 2026 to 15 July 2026** for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Swami Keshavanand Institute of Technology, Management & Gramothan, Jaipur.

The project presented in this report is based on the implementation developed during the training period. The project integrates a Streamlit frontend, FastAPI backend, SQLite database, Retrieval-Augmented Generation, semantic embeddings, FAISS vector search, Google Gemini, authentication, multimodal interaction and automated testing.

I further declare that this report has been prepared for academic purposes and represents my understanding and implementation of the project.

Nishant

Nishant Kumawat

23ESKCY036

Date: 08-10-2026

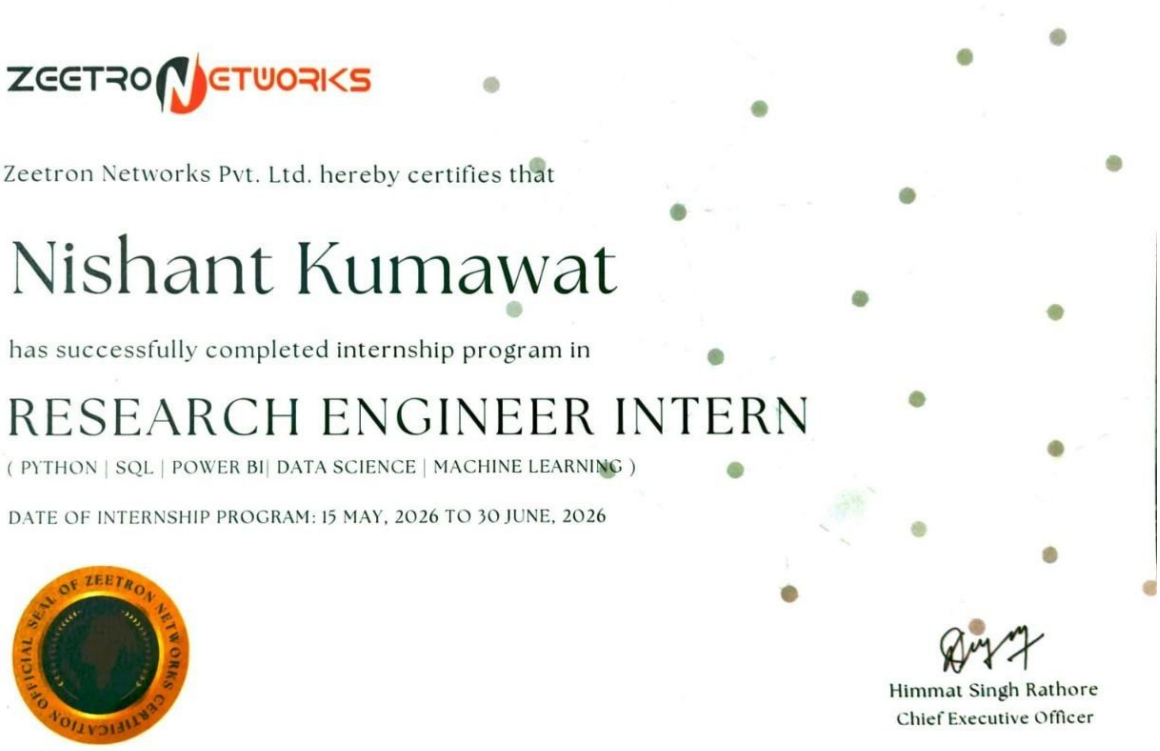
Certified that the above statement made by the student is correct to the best of our knowledge and belief

Examined by:

Mr. Shivank Lawaniya

Head of Department

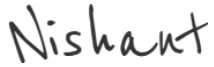
CERTIFICATE



ACKNOWLEDGEMENT

First and foremost, I wish to express my sincere thanks and gratitude to my esteemed Mentor "**Mr. Mohit Mahi**" who has contributed so much for successful completion of my Industrial Training by his thoughtful reviews and valuable guidance.

Next, I would like to tender my sincere thanks to "Prof. Name of HOD" (Head of Computer Science & Engineering) for his co-operation and encouragement. I am thankful to **Mr. Shivank Lawaniya**, Coordinator (VI-year internship 2026-27) for his support, cooperation and motivation provided during the internship for constant inspiration, presence and blessings.



Nishant Kumawat

23ESKCY036

About The Company

Zeetron Networks Pvt. Ltd. is a Jaipur-based IT and software company offering web and app development, digital marketing, networking solutions, cybersecurity, server migration, and IT infrastructure services. It also provides networking products, electronic accessories, cables, and CCTV solutions.

The company conducts workshops, internships, and job-oriented training in areas such as networking, cybersecurity, operating systems, databases, web technologies, and cloud infrastructure. Zeetron focuses on delivering customized, reliable, and cost-effective technology solutions while prioritizing quality, customer satisfaction, and timely delivery.

Table of Contents

1. Introduction……..……..1

1.1 Key Benefits……...……..**1**
1.2 Project Overview..……**1**
1.3 Project Scope…….....**2**
1.4 Intended Users……...…...**3**
1.5 Tools and Technologies used…….……...……**4**

2. Tools and Technologies…….…….……...……...…….5

2.1 Python Programming Language…….……..……**5**
2.2 Streamlit……...……...……**6**
2.3 Fast Api……..……..……**7**
2.4 Google Gemini ……...…….……**8**
2.5 RAG…….……**8**
2.6 SOLite Database…….……..**9**
2.7 JWT Authentication and Password Security **10**
2.8 PDF Processing **10**
2.9 Voice Processing **11**
2.10 Image and Vision Processing **11**

3. Dependencies……...…..12

3.1 Assumptions and Dependencies……...……..**13**
3.2 Hardware Requirements……**15**
3.3 Software Requirements……..**15**

4. Snapshots……...17

4.1 Login and Registration **17**
4.2 Admin Dashboard **17**
4.3 User Info **18**
4.4 Chat Page **18**
4.5 Side Navigation **18**
4.6 PDF Input and Processing **19**
4.7 PDF or Normal Conversation **19**

5. Implementation and Testing……...…….20

5.1 Implementation……...……...**20**
5.2 Testing……...…….……**24**

6. Result, Conclusion and Future Scope……..……27

7. References……...…….……..…….29

List of Figures

1. Login and Registration 17
2. Admin Dashboard 17
3. User Information 18
4. Chat Page 18
5. PDF Input and Processing 19
6. PDF or Normal Conversation 19

INTRODUCTION

1.1 Key Benefits:

The AI Research Assistant provides several important benefits:

1. Document-Grounded Answers

The system retrieves relevant information from uploaded documents before generating and answer. This helps the model provide responses based on the available document content.

2. Personalized User Workspace

Each user has a separate workspace containing their uploaded documents, vector data and conversation history.

3. Semantic Search

Instead of searching only for exact keywords, the system uses semantic embedding to identify content that is conceptually related to the user's question.

4. Conversation History

Users can save and access previous conversations, allowing them to continue their research sessions later.

5. Multiple AI Modes

The application supports document-based questions, PDF-question solving and general knowledge queries.

6. Multimodal Interaction

The system supports image-based questions and additional AI capabilities such as image generation.

7. Voice Interaction

Users can interact with the system using speech input and can also listen to generated responses.

8. Authentication and Security

The system provides user registration and login functionality. Passwords are securely hashed and authenticated sessions are maintained using JWT.

1.2 Project Overview:

The **AI Research Assistant** is an intelligent document-question-answering platform developed using modern Generative AI and Retrieval-Augmented Generation technologies.

The system allows users to upload PDF documents and ask questions about their content in natural language.

The overall workflow can be summarized as:

User → Upload PDF → Extract Text → Chunk Text → Generate Embedding → Store Vectors → User Query → Semantic Search → Retrieve Relevant Context → Gemini → Generated Answer

When a PDF is uploaded, the system extracts text from individual pages. The extracted text is divided into smaller overlapping chunks. Each chunk is converted into a numerical vector using the all-MiniLM-L6-v2 embedding model.

These vectors are stored in a FAISS index. When the user asks a question, the question is converted into an embedding and compared against the stored document embedding.

The most relevant chunks are then provided to the Gemini model as context. Gemini uses this context to generate a meaningful answer for the user.

1.3 Project Scope:

The project covers the development of an AI-powered document research assistant with the following functionalities:

- User registration and login
- JWT-based authentication
- PDF document upload

- PDF text extraction
- Text chunking
- Semantic embeddings
- FAISS vector search
- Retrieval-Augmented Generation
- Gemini-based answer generation
- Conversation creation and storage
- Conversation history
- Document management
- PDF-question solving
- Image-based question answering
- Voice input
- Voice output
- AI image generation
- Administrator portal
- Automated testing

The current implementation mainly focuses on text-based PDF documents. OCR-based processing of scanned documents is considered a future enhanceme

1.4 Intended Users:

The system can be used by different categories of users.

1. Students

Students can upload notes, assignments, research papers and examination material and ask questions about them.

2. Researchers

Researchers can use the application to quickly locate and understand information from research documents.

3. Faculty and Trainers

Faculty members can use the system to interact with teaching material and training documents.

4. General Users

Users can upload personal documents and ask questions about their content.

5. Administrators

Administrators can use the administrative interface to monitor application statistics and user-related information.

1.5 Tools and Technologies:

Technology	Purpose
Python	Main programming language
Streamlit	Frontend interface
FastAPI	Backend REST API
Google Gemini	Generative AI
LangChain	AI prompt/model integration
Sentence Transformers	Text embeddings
FAISS	Vector similarity search
SQLite	Database
JWT	Authentication
scrypt	Password hashing
pypdf	PDF text extraction
gTTS	Text-to-speech
SpeechRecognition	Speech-to-text
NumPy	Numerical operations
Python-dotenv	Environment configuration

TOOLS AND TECHNOLOGIES

The development of the AI Research Assistant required the integration of several technologies from the fields of web development, artificial intelligence, natural language processing, databases and information retrieval. The system is designed as a full-stack application in which the frontend provides an interactive environment for users while the backend manages authentication, document processing, retrieval, database operations and communication with generative AI services.

The major technologies used in the project include Python, Streamlit, FastAPI, Google Gemini, LangChain, Sentence Transformers, FAISS and SQLite. Additional libraries are used for PDF processing, voice interaction, image processing and authentication.

2.1 Python Programming Language:

Python is the primary programming language used throughout the project. It provides the foundation for the backend services, AI processing, document handling, database operations, vector processing and automated testing.

Python was selected because it has a large ecosystem of libraries for artificial intelligence and web development. The project requires several operations such as extracting text from PDF files, generating embeddings, performing similarity searches, communicating with Gemini, managing databases and processing user requests. These tasks can be implemented efficiently using Python.

2.1.1 Key Features of Python Used:

The major Python capabilities used in the project include:

- Modular programming
- File handling
- Database connectivity
- API development
- Exception handling
- Data processing
- AI model integration

- Vector processing
- Authentication
- Automated testing

Python also allows the project to maintain separate service modules rather than placing all functionality in one large application file.

2.1.2 Role in the Project:

Python is used in almost every layer of the application. The FastAPI backend is written in Python, the Streamlit interface is Python-based, and the RAG pipeline uses Python libraries for PDF processing, embeddings and vector retrieval.

The overall role of Python can therefore be represented as:

Python → Backend + AI Services + Document Processing + Database + Testing

2.2 Streamlit:

Streamlit is used to develop the interactive frontend of the AI Research Assistant.

One of the main advantages of Streamlit is that it allows web interfaces to be created using Python without requiring a separate frontend framework. This was suitable for the project because the application contains several AI-oriented controls and does not require a highly complex conventional web interface.

2.2.1 Features of Apache Airflow:

Streamlit provides:

1. Rapid development
2. Easy integration with Python
3. Interactive widgets
4. File-upload components
5. Session management
6. Simple deployment
7. Easy integration with AI applications

2.2.2 Frontend Workflow:

The frontend receives the user's input and communicates with the FastAPI backend.

USER

↓

STREAMLIT INTERFACE

↓

HTTP REQUEST

↓

FASTAPI BACKEND

↓

AI / DATABASE SERVICES

↓

RESPONSE

↓

STREAMLIT INTERFACE

2.3 FastAPI:

FastAPI is used as the backend framework of the project.

The backend provides REST API endpoints through which the Streamlit application communicates with the core services. Instead of performing all processing directly in the frontend, the application sends requests to FastAPI.

The backend handles:

- Authentication
- User registration
- Login
- Document uploading
- Document management
- Chat
- Conversation history

- Voice services
- Vision services
- Image generation
- Administrative operations

2.3.1 Benefits of FastAPI:

FastAPI was selected because it provides:

- High-performance API development
- Request validation
- Automatic API documentation
- Dependency injection
- Modular route organization
- Python compatibility
- Easy integration with authentication

The backend also uses dependency-based authentication so that protected operations can determine the currently authenticated user.

2.4 Google Gemini:

Google Gemini is used as the generative AI layer of the system.

The model receives the user's question and, where appropriate, relevant information retrieved from the user's uploaded documents. It then generates a natural-language response.

The current project configuration uses Gemini 2.5 Flash as the default model, with a separately configured image model for image generation.

2.4.1 Role in the Project:

Gemini is used for:

- Natural-language response generation
- RAG-based question answering
- PDF question solving
- Image-based question answering

- AI conversation
- Image generation

2.5 Retrieval-Augmented Generation:

Retrieval-Augmented Generation is the core concept behind the document-question-answering functionality.

A conventional chatbot can generate answers based on the knowledge available to the model. However, a research assistant must be able to work with private documents supplied by individual users.

RAG addresses this requirement by combining:

Information Retrieval + Generative AI

The retrieval component finds relevant information, while the generative model converts that information into a readable response.

.

2.5.1 Role:

The project follows these major stages:

1. PDF upload
2. Text extraction
3. Text chunking
4. Embedding generation
5. Vector storage
6. User question
7. Query embedding
8. Similarity search
9. Relevant context retrieval
10. Prompt construction
11. Gemini response generation

2.6 SQLite Database:

SQLite is used as the persistent database of the application.

It stores application information such as:

- User accounts
- Conversations
- Messages
- Document-related information
- Metadata

SQLite is suitable for this project because it is lightweight and does not require a separate database server.

The database also uses foreign keys and indexes to support user isolation and efficient history queries. Migration logic is included as part of database initialization.

2.7 JWT Authentication and Password Security:

Security is an important requirement because users may upload private documents and maintain personal conversation histories.

The application uses:

- JWT authentication
- Password hashing
- Expiration-based tokens
- Protected API dependencies
- Separate administrator credentials

Passwords are processed using Python's scrypt implementation with a random salt. After successful login, a signed JWT is generated with an expiration time.

2.8 PDF Processing:

The project uses pypdf for PDF text extraction.

When a PDF is uploaded, its pages are processed and text is extracted. The extracted information is then passed to the chunking and embedding stages.

The application maintains information such as source filename and page number so that document context can remain associated with the retrieved content.

The current version assumes that the PDF contains extractable text. OCR for scanned PDFs is not included in the current implementation.

2.9 Voice Processing:

The application also includes voice functionality.

Two important libraries are:

- SpeechRecognition
- gTTS

SpeechRecognition is used to convert supported speech input into text, while gTTS converts generated text into speech.

The generated response is cleaned before speech conversion so that unwanted formatting such as citations or markdown does not interfere with speech output.

2.10 Image and Vision Processing:

The system provides image-based question answering.

Users can submit supported image formats and ask questions related to the uploaded image.

The vision service validates supported image signatures before sending the content to Gemini.

Supported image types include formats such as:

- PNG
- JPEG
- GIF
- WEBP

The application also contains a separate image-generation capability.

DEPENDENCIES

The AI Research Assistant depends on several software packages, system resources, external services and configuration variables. Correct installation and configuration of these dependencies is necessary for the successful execution of the application.

The project is designed to run on common operating systems and uses a Python virtual environment to isolate project dependencies.

3.1 Assumption and Dependency:

The design and implementation of the project are based on a few key assumptions regarding the system environment, tools, and resources. These assumptions serve as the foundation for the smooth operation of the project.

3.1.1 Assumptions:

The following assumptions are made for the application:

1. A stable internet connection is available.
2. A valid Google API key is configured.
3. Required Python packages are installed.
4. The FastAPI backend is running before the Streamlit frontend.
5. Sufficient RAM is available for embedding generation and vector processing.
6. Sufficient storage is available for uploaded documents and vector data.
7. Uploaded PDFs contain extractable text.
8. Required environment variables are correctly configured.

These assumptions are consistent with the project documentation.

3.2 Backend Dependencies:

The backend uses several packages for API development and security.

- **FastAPI**

  Used for creating REST API routes.

- **Uvicorn**

  Used to run the FastAPI application.

- **PyJWT**

  Used for creating and validating JSON Web Tokens.

- **python-dotenv**

  Used to load configuration values from environment variables.

- **python-multipart**

  Used for handling file uploads.

3.3 RAG Dependencies:

The RAG pipeline requires:

- pypdf
- NumPy
- FAISS
- Sentence Transformers
- LangChain Core
- LangChain Text Splitters
- Google GenAI components

These libraries collectively perform document extraction, text processing, embedding generation, retrieval and AI generation.

3.4 Frontend Dependencies:

The primary frontend dependency is:

Streamlit

The frontend also communicates with the backend using HTTP requests.

The interface is responsible for collecting user input and presenting backend responses.

3.5 Database Dependencies:

SQLite is used for persistent storage.

The database is responsible for storing user and conversation-related information.

The application uses database relationships and indexes to support efficient access and user isolation.

3.6 Voice Dependencies:

The voice functionality requires:

- **SpeechRecognition**

Used for speech-to-text conversion.

- **gTTS**

Used for text-to-speech generation.

Voice services may require internet connectivity depending on the service configuration.

3.7 Image Dependencies:

The vision functionality uses Google GenAI capabilities to process image content.

The system validates the image before sending it to the AI service.

This validation is important because unsupported or malformed files should not be processed.

3.8 Hardware Requirements:

3.8.1 Minimum Practical Configuration:

- Processor: Modern dual-core or equivalent
- RAM: 8 GB
- Storage: At least 10 GB free
- Stable internet connection
- Modern web browser

3.8.2 Recommended Configuration:

- Modern quad-core processor
- 16 GB RAM
- SSD storage
- Stable broadband connection

The project documentation specifically identifies 8 GB RAM as practical for development and 16 GB as recommended when the embedding model, backend, frontend and browser are running together.

3.9 Software Requirements:

The project can run on:

- Windows
- Linux
- macOS

The recommended environment includes:

- Python
- Virtual environment
- pip
- Streamlit
- FastAPI
- Uvicorn
- Required Python packages
- Modern browser

3.14 Environment Configuration:

The project uses environment variables for configuration.

Important configuration values include:

- GOOGLE_API_KEY
- GEMINI_MODEL
- Database path
- Vector-store path
- Upload limits
- Image settings

Sensitive credentials should not be directly hard-coded into the source code.

Advantages

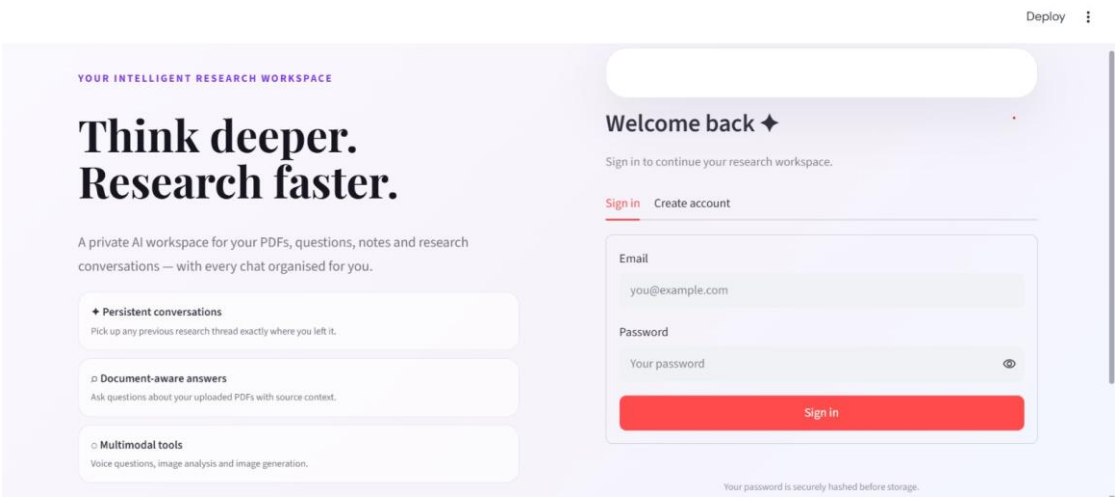
- Improved security
- Easy configuration
- Different settings for development and production
- Prevention of accidental credential exposure

SNAPSHOTS

This chapter presents the visual representation of the AI Research Assistant. Screenshots should be inserted into this chapter to demonstrate the working interface and major features of the application.

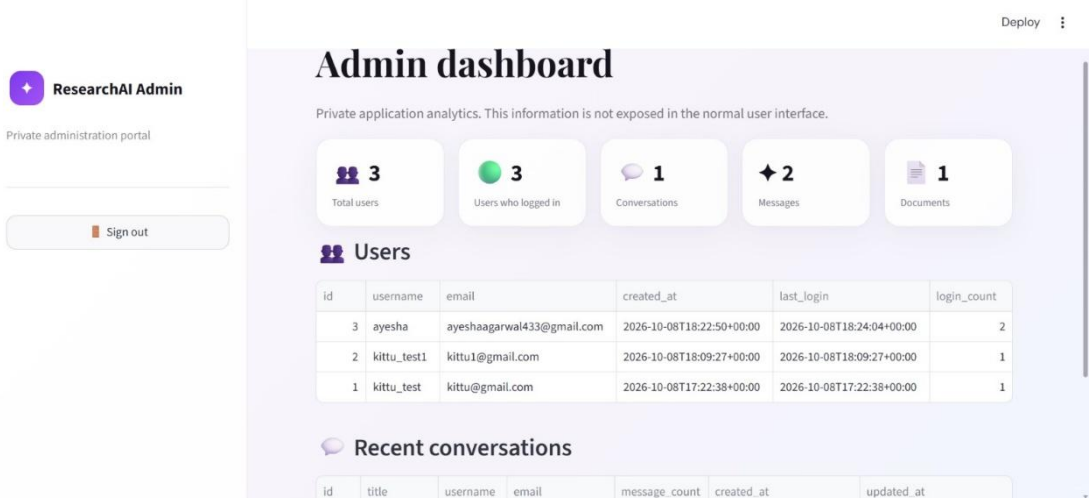
The project archive primarily contains source code and implementation rather than a finalized screenshot collection. Therefore, the final report should include screenshots captured directly from the running application. The implementation-derived architecture and workflow are already documented in the project report.

4.1 Login and Registration:



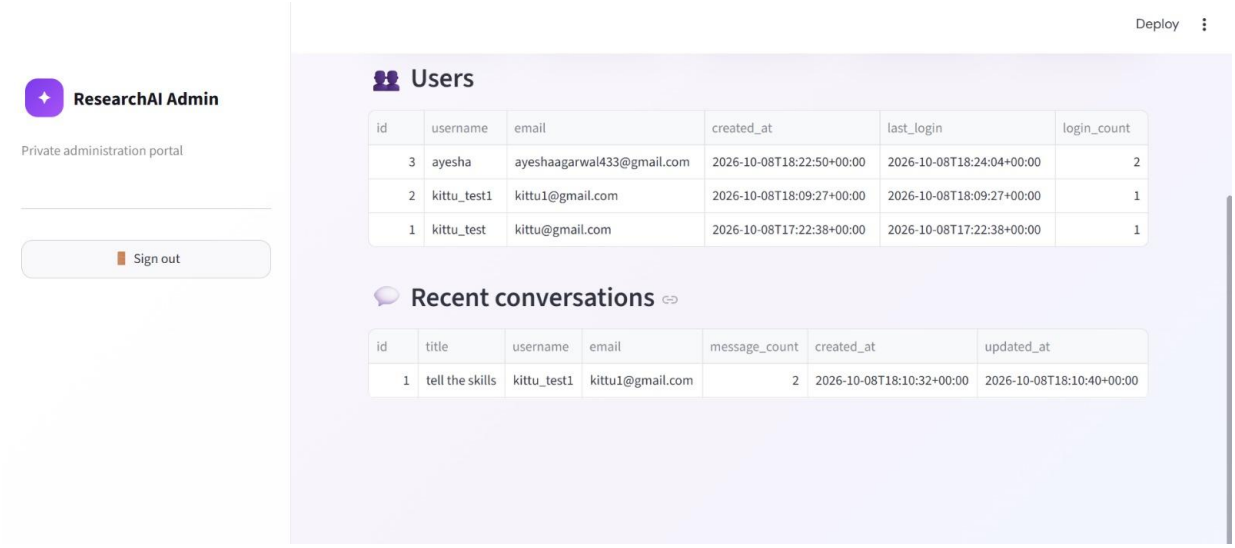
1. Login and Registration

4.2 Admin Dashboard:



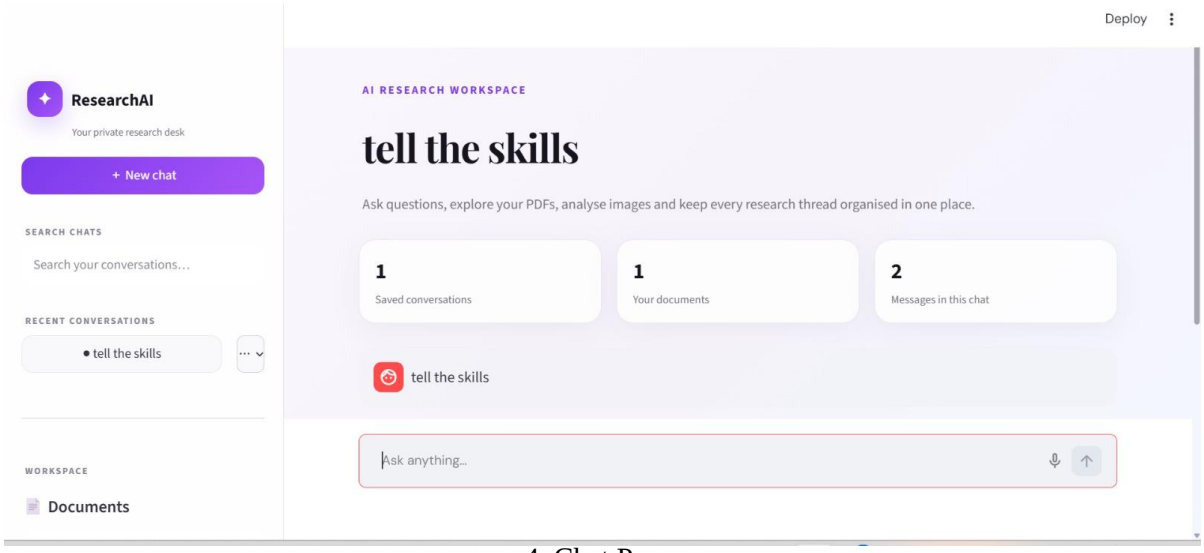
2. Admin Dashboard

4.3 User Info:



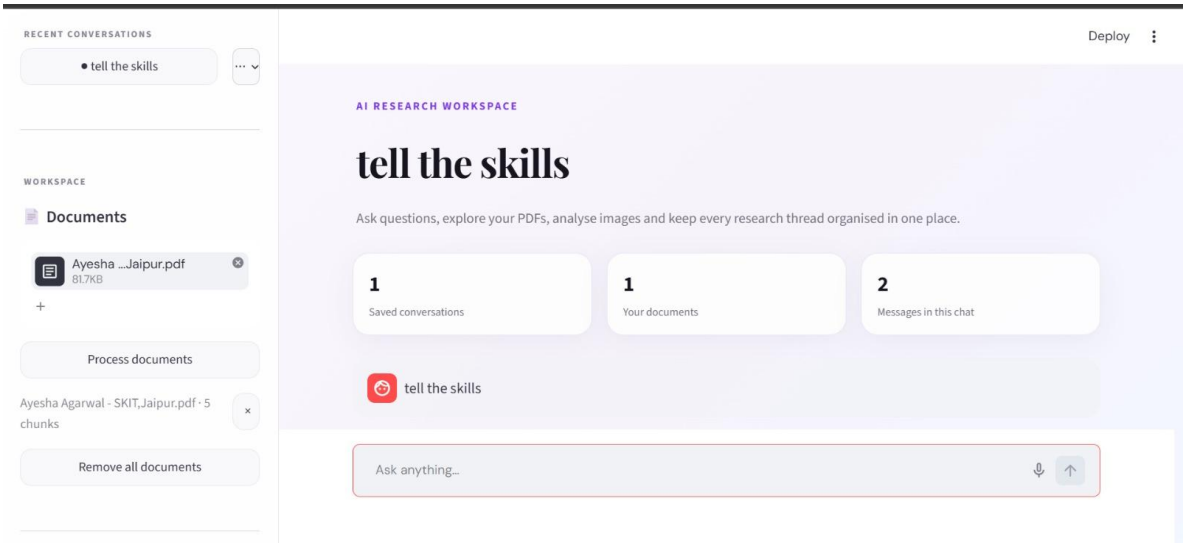
3. User Info

4.4 Chat Page:

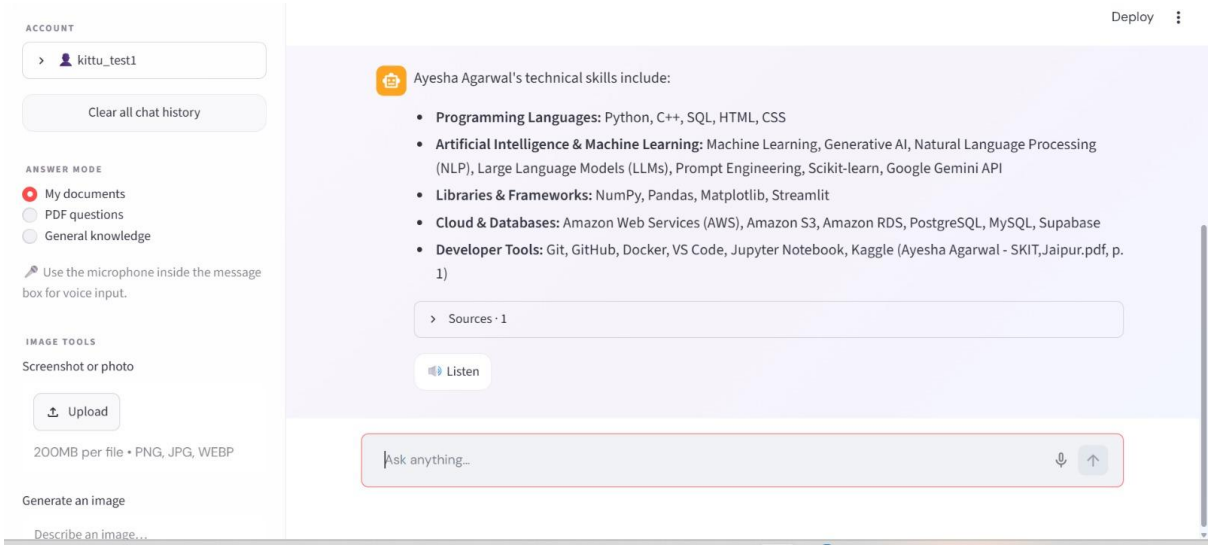


4. Chat Page

4.5 Side Navigation:



5. pdf input and processing



6. pdf or normal conversation

IMPLEMENTATION AND TESTING

Implementation involves converting the proposed architecture into a functional application. The AI Research Assistant was implemented as a modular system consisting of frontend, backend, database, document-processing, retrieval and AI-generation components.

The primary workflow is:

Authenticate → Upload → Extract → Chunk → Embed → Store → Retrieve → Generate → Save Conversation → Display Answer.

5.1 System Architecture:

The system consists of several major layers.

- **Presentation Layer**

  Implemented using Streamlit.

- **API Layer**

  Implemented using FastAPI.

- **Persistence Layer**

  Implemented using SQLite and per-user vector storage.

- **Retrieval Layer**

  Implemented using Sentence Transformers and FAISS.

- **AI Layer**

  Implemented using Google Gemini.

  The separation of these layers improves maintainability.

5.2 User Authentication:

The first step is authentication.

A new user registers by providing the required account information.

The password is converted into a secure hash using scrypt before being stored.

During login:

1. User enters credentials.
2. Backend retrieves the stored account.
3. Password is verified.
4. JWT is generated.
5. Token is used for authenticated requests.

The application also uses token expiration to reduce security risks

5.3 PDF Upload:

The user can upload a PDF through the Streamlit interface.

The backend performs validation before processing.

Validation includes:

- File type
- File size
- Duplicate filename checks
- File accessibility

If validation succeeds, the document is passed to the PDF-processing service.

5.4 PDF Text Extraction:

The pypdf library is used to extract text from the uploaded PDF.

The system processes the document page by page.

The extracted data can retain:

- Source filename

- Page number
- Extracted text

This information is useful when retrieved chunks need to be associated with their original document location.

5.5 Text Chunking:

Large documents cannot always be processed as a single block.

Therefore, extracted text is divided into smaller chunks.

The project uses approximately:

- **Chunk size:** 1000 characters
- **Overlap:** 200 characters

The overlap ensures that important information near the boundary between two chunks is not completely separated.

5.6 Vector Storage:

FAISS is used to store and search the embeddings.

The vector store is maintained separately according to the user so that one user's documents are not mixed with another user's information.

This is an important part of privacy and user isolation.

5.7 Query Processing:

When the user asks a question, the question is processed using the same embedding model.

The retrieved chunks are then passed to the RAG service.

The retriever also ensures that the requested number of results does not exceed the number of available chunks.

User Question

↓

Query Embedding

↓

FAISS Similarity Search

↓

Top Relevant Chunks

5.8 Prompt Construction:

The retrieved information is combined with the user's question.

Recent conversation history may also be included to maintain context.

System Instructions

+

Retrieved Context

+

Recent Conversation

+

Current Question

↓

Prompt

5.9 Vision Processing:

The vision service allows the user to ask questions about images.

Before processing, the system validates supported image formats.

Image Upload

↓

Format Validation

↓

Vision Request

↓

Gemini

↓

Answer

5.10 Image Generation:

The application also supports AI-based image generation.

The user provides a description, which is passed to the configured image-generation service.

The generated image is then returned to the application.

This feature demonstrates the extension of the research assistant from text-based interaction toward multimodal AI.

5.11 Testing Strategy:

Testing was performed at multiple levels to verify that individual components and integrated workflows behaved correctly.

The main testing categories were:

1. Security testing
2. Database testing
3. Vector-store testing
4. Retrieval testing
5. PDF testing
6. PDF-question testing
7. Voice testing
8. Vision testing

5.12 Security Testing:

Security tests verify:

- Password hashing
- Password verification
- JWT generation
- JWT expiration
- Token tampering detection
- User authentication
- Administrator authentication

These tests are important because authentication controls access to private documents and conversations.

5.13 Database Testing:

Database tests verify:

- User creation
- Duplicate email rejection
- Message ordering
- Source persistence
- User isolation
- Conversation clearing

These tests ensure that information belonging to one user does not incorrectly appear in another user's workspace.

5.14 Voice and Vision Testing:

Voice testing includes:

- Empty speech handling
- Citation cleaning
- Markdown cleaning

Vision testing verifies:

- PNG recognition
- JPEG recognition
- GIF recognition
- WEBP recognition
- Prompt construction

These tests help ensure that multimodal features behave correctly.

5.15 Error Handling:

Error handling is implemented for several situations:

- Missing API key
- Invalid session
- Expired JWT
- Oversized upload
- Unsupported image
- Password-protected PDF
- Scanned PDF
- AI model failure
- Image-generation failure

Instead of allowing these errors to terminate the complete application, controlled responses are provided.

5.26 Security Measures:

The project implements several security measures:

- Secure password hashing
- JWT expiration
- Protected API routes
- Separate administrator credentials
- Separate administrator signing key
- User-isolated vector data
- Sanitized upload filenames
- Exclusion of password hashes from public metadata

These mechanisms improve the overall security of the application.

RESULT, CONCLUSION AND FUTURE SCOPE

6.1 Result:

The **AI Research Assistant** successfully provides an interactive platform for document-based question answering using Retrieval-Augmented Generation (RAG). The system supports PDF uploading, text extraction, semantic search, AI-generated responses, conversation history, authentication, voice interaction and image-based question answering. FAISS enables relevant document retrieval, while Google Gemini generates contextual responses. The project demonstrates the practical integration of **Generative AI, RAG, APIs, databases and vector search** into a single application.

6.2 Conclusion:

The AI Research Assistant successfully demonstrates the integration of Retrieval-Augmented Generation with standard software engineering practices.

The project combines a Streamlit frontend, FastAPI backend, SQLite database, semantic embeddings, FAISS vector search and Google Gemini into a unified application.

The system enables users to upload documents and interact with them through natural-language questions. The RAG architecture retrieves relevant information before generating responses, making the application suitable for research and study-oriented use cases.

The project also extends beyond basic question answering through authentication, conversation history, PDF question solving, voice interaction, image question answering and image generation.

From a software engineering perspective, the project demonstrates the importance of modular architecture, API-based communication, database persistence, authentication and automated testing.

Overall, the project provided practical exposure to modern AI application development and demonstrated how generative AI can be combined with information retrieval and conventional software engineering to build a useful real-world application.

6.3 Future Scope:

The project can be further enhanced by:

- OCR support for scanned PDFs.
- Support for DOCX, TXT, Excel and PPT files.
- Cloud deployment for multiple users.
- Managed vector databases for better scalability.
- Streaming responses for faster interaction.
- Multilingual text and voice support.
- Better processing of tables, diagrams and images in PDFs.
- RAG evaluation using accuracy, relevance and faithfulness metrics.
- Role-based access control and improved monitoring.

REFERENCES

2 Python Software Foundation, Python Documentation.
3 FastAPI, FastAPI Official Documentation.
4 Streamlit, Streamlit Official Documentation.
5 Google AI, Gemini and Google GenAI Documentation.
6 LangChain, LangChain Official Documentation.
7 Sentence Transformers, Sentence Transformers Documentation.
8 FAISS, Facebook AI Similarity Search Documentation.
9 SQLite, SQLite Official Documentation.
10 PyJWT, JSON Web Token Documentation.
11 pypdf, Python PDF Processing Documentation.
12 SpeechRecognition, Python Speech Recognition Documentation.
13 gTTS, Google Text-to-Speech Python Library Documentation.
14 NumPy, NumPy Documentation.
15 Submitted Project Archive, AI_Research_assistant_v7_admin.zip.
16 Supplied Reference Report, Industrial Training Report 2025.